

NetMonitor

Propósito e arquitetura

NetMonitor é uma plataforma independente de monitoramento de infraestrutura, separada do Dashboard principal da clínica, localizada em `C:\NetMonitor`. Monitora: chamados do GLPI, páginas web (portais de convênios), servidores da rede local e o link de internet.

- **Backend:** FastAPI (`C:\NetMonitor\backend\main.py`), rodando na porta 8700.
- **Persistência:** SQLite (`C:\NetMonitor\backend\monitor.db`), sem dependência do banco SQL Server `SMART` usado pelo Dashboard.
- **Frontend:** React + Vite (`C:\NetMonitor\frontend\src\App.jsx`), servido como build estático (`frontend/dist`) montado pelo próprio FastAPI (rota catch-all em `main.py` , que também bloqueia acesso direto a `.env` e `monitor.db` via URL).
- **WhatsApp:** não tem sessão própria — reaproveita a sessão WPPConnect **já autenticada pelo Dashboard** (leitura somente, nunca escreve na config do Dashboard). Os números que recebem alerta de infraestrutura são configurados separadamente (`ALERTA_WHATSAPP_NUMEROS` no `.env` do NetMonitor), independentes de quem recebe os resumos de produção do Dashboard.

Tipos de monitor

Tabela `monitores` (SQLite): `id, tipo, nome, alvo, intervalo_segundos, ativo, config_extra` (JSON), `criado_em`.

- **http** (`checar_http`): faz `GET` na URL (`alvo`), com `follow_redirects=True`. Considera **up** se `200 <= status < 400`. Timeout de `TIMEOUT_HTTP_SEGUNDOS = 240` (4 minutos) — só marca como "caiu" se a página realmente não responder nesse tempo (subiu de 10s originalmente, depois 30s, hoje 4min, a pedido do usuário, pra evitar falso alarme por lentidão pontual). Se o host for privado (IP privado ou `.local`, via `_host_eh_privado`), pula a verificação de certificado SSL (equipamento de rede com certificado autoassinado).
- **servidor** (`checar_servidor`): ping via utilitário `ping` do Windows (não precisa de privilégio elevado, diferente de socket ICMP puro), timeout de 1s por tentativa.
- **internet** (`checar_internet`): monitor fixo (criado automaticamente por `inicializar_db`, não pode ser removido — `alvo` default `"1.1.1.1,8.8.8.8"`). Considera **up** se qualquer um dos IPs da lista responder ao ping.
- **glpi** (`checar_glpi`): fluxo REST da API do GLPI — `initSession` (com User-Token) → `search/Ticket` (filtra status "morethan 0", ou seja campo `12 > 0`) → `killSession`. Considera **up** se a API responder, com o detalhe mostrando quantos chamados estão abertos (status em `GLPI_STATUS_ABERTO = {1,2,3,4}` = Novo, Em atendimento atribuído, Em atendimento planejado, Pendente — exclui Solucionado(5) e Fechado(6)).

Scheduler — modelo de concorrência

`scheduler.py` roda em thread de background (`iniciar_scheduler_em_background`, chamada no startup do `main.py`), em ciclo de **15 segundos** (`_CICLO_SEGUNDOS`).

A cada ciclo: 1. `monitores.monitores_pendentes()` retorna os monitores ativos cujo `intervalo_segundos` já venceu desde a última checagem (e que não estejam já em execução — ver `guard` abaixo). 2. **Todas as checagens pendentes do ciclo são submetidas de uma vez** a um `ThreadPoolExecutor(max_workers=8)`, não uma de cada vez — decisão explícita: como o timeout HTTP pode chegar a minutos, processar sequencialmente faria uma página lenta atrasar a checagem de todos os outros monitores atrás dela na fila. 3.

Cada `future.result()` é aguardado com timeout de `_TIMEOUT_CHECAGEM_SEGUNDOS = TIMEOUT_HTTP_SEGUNDOS + 10` (250s) — folga de 10s sobre o timeout HTTP interno. Se estourar, loga erro mas não trava o ciclo.

Guard contra checagem duplicada em andamento: como uma checagem HTTP pode ficar rodando por até 4 minutos, um monitor pode continuar "pendente" (não teve checagem nova registrada) em múltiplos ciclos de 15s enquanto a anterior ainda roda. Um `set()` global `_em_execucao` (protegido por `threading.Lock`) registra monitores com checagem em andamento — `monitores_pendentes()` pula quem já está no set, e `executar_checagem()` também verifica o guard no início e retorna `{"status": "skipped", ...}` sem reprocessar se detectar duplicata.

Lógica de alertas (WhatsApp)

`_avaliar_alerta()` em `monitores.py`, chamada ao final de toda `executar_checagem()`. Dispara mensagem via `alertas.enviar_alerta_whatsapp()` **somente** nestas transições (a pedido explícito do usuário — "latência alta" não gera mais alerta, fica só registrada no histórico):

- **Caiu** (`status_atual == "down"` e anterior não era down): 🚫 `NetMonitor - {nome} caiu! {alvo}`
- **Voltou ao ar** (`status_atual == "up"` e anterior era down): 🟢 `NetMonitor - {nome} voltou ao ar. Ficou {duração} fora do ar. {alvo}` — a duração é calculada por `_duracao_queda()`, que busca a última checagem `up` registrada antes do timestamp atual e subtrai; formatada por `_formatar_duracao()` (min, ou `XhYYmin` acima de 1h).

Detecção de novo chamado GLPI (`detectar_novos_chamados_glpi`, chamada de dentro de `executar_checagem` quando `tipo == "glpi"` e status é `up`): compara os chamados abertos agora com os já vistos (tabela `glpi_chamados_vistos`, chave primária `chamado_id`). Chamados não vistos antes disparam 📞 `NetMonitor • GLPI - Novo chamado #{id} {titulo}` e são inseridos na tabela de vistos. **Importante:** na primeiríssima execução (tabela `glpi_chamados_vistos` vazia), a função apenas semeia o histórico atual sem enviar alerta nenhum — evita que todo chamado já aberto no GLPI seja anunciado de uma vez só quando o recurso é ativado.

`alertas.enviar_alerta_whatsapp()`: lê a sessão/token do WPPConnect direto do arquivo `C:\Dashboard\backend\whatsapp_config.json` (somente leitura), envia via `POST {wppconnect_url}/api/{session}/send-message` com `Authorization: Bearer {token}`, para cada número em `ALERTA_WHATSAPP_NUMEROS` (com 1s de espera entre envios). Se o token estiver expirado (HTTP 401), retorna erro descritivo — não tenta renovar (quem renova é o próprio scheduler do Dashboard, periodicamente, para uso dele).

Baseline de latência e detecção de anomalia

`calcular_baseline_latencia(monitor_id, limite_amostras=200)`: média (`statistics.mean`) e desvio padrão populacional (`statistics.pstdev`) do `tempo_resposta_ms` das últimas 200 checagens com `status='up'` (down não tem latência real comparável). Só calcula se houver pelo menos `MIN_AMOSTRAS_BASELINE = 10` amostras (evita baseline instável com histórico curto). `limite_normal_ms = média × FATOR_ALERTA` (`FATOR_ALERTA = 1.8`) — ou seja, latência é considerada "alta" quando ultrapassa 1,8× a média histórica. Usado tanto no card de status (`/api/status`, campo `latencia_alta`) quanto na linha de referência do gráfico de histórico (`/api/historico/{id}`) e no cálculo de incidentes.

Feature "Incidentes" (`/api/incidentes`)

`calcular_incidentes(monitor_id, desde, ate)` reconstrói, a partir do histórico bruto de `checagens`, os episódios de indisponibilidade e os desvios de latência no período — os mesmos eventos que gerariam alerta no WhatsApp, mas agregados para o período inteiro:

- Busca a checagem imediatamente anterior ao início do período (`anterior_row`) para saber o estado de entrada (se já estava "down" ao entrar no período, considera que a queda começou exatamente em `desde`).

- Percorre as checagens do período em ordem: cada transição para "down" conta uma queda (`quedas += 1`) e marca `queda_inicio` ; a transição de volta para "up" soma a duração daquele episódio a `tempo_indisponivel` . Se ainda estiver "down" no fim do período, conta o tempo até `ate` .
- `uptime_pct = (1 - tempo_indisponivel / duração_total_do_período) × 100` .
- `desvios_latencia` : conta transições para "alta" (latência acima do `limite_normal_ms` do baseline) que não estavam já altas na checagem anterior — mesmo critério de transição usado nos outros contadores, evita contar a mesma anomalia repetidamente enquanto ela persiste.

Endpoint `/api/incidentes?horas=168` (padrão 7 dias) roda esse cálculo para todos os monitores ativos e ordena do pior para o melhor (mais tempo indisponível primeiro, depois mais quedas).

Frontend (`PainelIncidentes` / `CardIncidente` em App.jsx): aba " Incidentes" com seletor de período (24h / 7 dias / 30 dias), grid de cards — um por monitor — mostrando uptime %, nº de quedas, tempo total fora do ar (formatado em min ou h) e nº de desvios de latência, com a cor da borda esquerda mudando conforme a gravidade (verde = 0 quedas, amarelo ≤ 3, vermelho acima disso).

Outros endpoints

- `GET /api/status` : status atual de todos os monitores ativos (última checagem + baseline + flag `latencia_alta`) — alimenta a aba "Monitores".
- `GET /api/historico/{monitor_id}?horas=24` : série de checagens no período + baseline, para o mini-gráfico expansível de cada card.
- `POST /api/monitores` : cria monitor (tipo `http` / `servidor` / `glpi` ; token do GLPI vai em `config_extra` como JSON).
- `DELETE /api/monitores/{id}` : remove monitor e seu histórico (bloqueado para o tipo `internet` , que é fixo).
- `POST /api/monitores/{id}/checar-agora` : dispara uma checagem manual imediata (mesma função `executar_checagem` , fora do ciclo do scheduler).
- `GET /api/glpi/chamados` : lista os chamados abertos do primeiro monitor GLPI cadastrado (usado no detalhe expandido do card).
- `GET /api/health` : healthcheck simples, indica se o scheduler está disponível.

Frontend — aba "Monitores": cards de resumo (No Ar / Fora do Ar / Latência Alta / Total), formulário de criação de monitor, e os cards individuais agrupados por tipo (Internet, GLPI, Servidores, Páginas Web), cada um com botão de gráfico 24h, checagem manual e remoção.